

Introduction to Blockchain and Smart Contracts

Lecture6: Nakomoto Consensus, PoS, Bitcoin Scripts

Subodh Sharma | IIT Delhi | Feb 02, 2026



Miner Txn Selection

- **What are the block constraints?**
 - **Block weight limit:** bound on the number of vBytes!
 - **Ordering:** Parents must appear before children in the block
 - If txns are independent, **sort by fee-rate** (in the descending order)
 - **Fee-rate:** #satoshi/vByte
 - Otherwise, use **package-aware scoring**
 - **Package** = txn + unconfirmed ancestors required to make the txn valid in a block
 - $$\text{Pkg fee rate} = \frac{\sum(\text{fees of pkg})}{\sum(\text{weight of pkg})}$$

Forking Attack

- Miner sends some money to the victim, say, B
- B waits to see the txn included in an accepted block and offers the service to the miner
- Miner **privately** mines to exclude the earlier txn
 - Presumption: The miner has $>51\%$ hash power, it can mine a secret chain faster than the main-net
 - Miner bcasts the private chain and orchestrates the double-spending attack through a fork!

Selfish Mining

- **Intuition:** Ties in the chain create an opening for an attack
 - An attacker can engineer the ties s.t. some honest miners will mine on the attacker's branch, thus increasing the attacker's chance of winning the race
- **Step-by-step Attack scenario:**
 - Attacker, A, mines a block, say a_1 , and gets +1 lead from the tip of the longest chain
 - Attacker withholds and starts mining privately, trying to extend the chain
 - **Case 1:** Attacker again finds a block before the honest n/w; At this point attacker publishes to stay ahead
 - **Case 2:** Honest miners find a block, b_1 ; attacker immediately publishes, a_1 . Now γ fraction of miners may mine on a_1 and $1 - \gamma$ on b_1
 - Finally, in case 2, if the attacker side mines the next block, the attacker wins!

Selfish Mining

- **Threshold Formula:** $\alpha > (1 - \gamma)/(3 - 2\gamma)$
 - If $\gamma = 0$, then the attacker has NO advantage $\rightarrow$ requires $\alpha > 1/3$ of the total hash-rate of the n/w
 - If $\gamma = 0.5$, then $\alpha > 1/4$, which means with 25% hash-rate the attack can be orchestrated!
- **Mitigation:** Faster relay n/ws; compact block relay;

Proof of Stake

- Maintains a set of **validators**
- Validators have to **put at stake** some amount of currency
 - Stake is locked until the end of the round
- Bigger stakeholder has a **higher chance** of winning the competition
- Two types of PoS:
 - Chain-based PoS: Retains the longest-chain rule. Eg: Peercoin
 - BFT PoS: uses BFT consensus for finalising new blocks

Proof of Stake

Chain-based PoS

- Every validator's stake is frozen until the end of the round
- N/w performs a random exercise to select the “**generator**”
 - The generator generates the new block and claims the reward just like in PoW
 - The Generator is selected through a **verifiable random function**
 - A validator takes a seed and their Private key and produces a random number and a NIZK proof. If the random number $<$ Threshold (determined by the validator's stake), then the validator is selected.
- Other validators attest and vote; if sufficient votes are observed the block may be considered as finalised

Proof of Stake

Chain-based PoS

- **Security Issues:**
 - Stake-majority attack: **Rich get richer** — can influence what gets included on the chain and also rewrite history
 - Long-range attacks: An attacker obtains validator keys for far in the past and creates an alternative chain
 - Nothing-at-stake: In vanilla PoS, validators might sign multiple competing forks because it's cheap.
 - Randomness manipulation: PoS protocols rely on randomness to pick proposers/committees. If attackers can influence or predict randomness, they can increase the chance they (or their allies) are selected.
- **Mitigation:** Mostly by slashing staked principal
- **READING ASSIGNMENT: Understand BFT-based PoS protocol**

Bitcoin Deep Dive: Actual Bitcoin Txn

```
{
  "hash": "5a42590fbe0a90ee8e8747244d6c84f0db1a3a24e8f1b95b10c9e050990b8b6b",
  "ver": 1,
  "vin_sz": 2,
  "vout_sz": 1,
  "lock_time": 0,
  "size": 404,
  "in": [
    {
      "prev_out": {
        "hash": "3be4ac9728a0823cf5e2deb2e86fc0bd2aa503a91d307b42ba76117d79280260",
        "n": 0
      },
      "scriptSig": "30440..."
    },
    {
      "prev_out": {
        "hash": "7508e6ab259b4df0fd5147bab0c949d81473db4518f81afc5c3f52f91ff6b34e",
        "n": 0
      },
      "scriptSig": "3f3a4ce81...."
    }
  ],
  "out": [
    {
      "value": "10.12287097",
      "scriptPubKey": "OP_DUP OP_HASH160 69e02e18b5705a05dd6b28ed517716c894b3d42e OP_EQUALVERIFY OP_CHECKSIG"
    }
  ]
}
```

metadata

input(s)

output(s)

Bitcoin Deep Dive: The UTXO Model

- **UTXO:** Unspent Transaction Outputs
 - A bitcoin txn consumes prior outputs and creates new outputs
 - Each output is: $Out_j = (v_j, \text{scriptPubKey}_j)$
 - v_j is the amount in satoshi; scriptPubKey is a program (Script) that specifies the spending conditions
 - Each input references a specific previous output:
 - $in_i = (\text{prevout}_i, \text{scriptSig}_i, \text{sequence}_i, \text{witness}_i)$

Bitcoin Deep Dive: The Bitcoin Script

- **Script:** A non-Turing-complete language, stack based
- **Script basics:**
 - `OP_DUP` : duplicate top item
 - `OP_HASH160` : $\text{HASH160}(x) = \text{RIPEMD160}(\text{SHA256}(x))$
 - `OP_EQUALVERIFY` : check equality and fail if false
 - `OP_CHECKSIG` : verify ECDSA (or Schnorr under Taproot) signature against a public key and a signature-hash

-

Bitcoin Transaction Types

Pay to Public Key Hash

- **Key intuition:** Instead of locking funds to a full public key, you lock them to the hash of the key
- Locking script (scriptPubKey):

```
OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

- The unlocking script (scriptSig, to spend):

```
<Signature> <Full_PublicKey>
```

Bitcoin Transaction Types

Pay to Script Hash

- **Key intuition:** While P2PKH locked funds to a person's public key, P2SH locks funds to a set of rules (a script)
- The locking script (ScriptPubKey)

```
OP_HASH160 <20_Byte_RedeemScriptHash> OP_EQUAL
```

- The unlocking script (ScriptSig):

```
OP_0 <Sig_A> <Sig_C> <Serialized_RedeemScript>
```

- The serialised redeem script on getting unpacked will look like:

```
OP_2 <PubKey_A> <PubKey_B> <PubKey_C> OP_3 OP_CHECKMULTISIG
```

